Week 9 Milestone Report: Environment and Repository Check

Course: CS3320 Database Systems, Instructor Dr. Chibuzo Ukegbu
Student: Shritej Gunda (individual project)
Project: Factory Sherlock: Manufacturing Simulation, Quality Control, and Defect Investigation on MySQL
Date: [submission date]
Repository: https://github.com/shritej6/factory-sherlock
Commit or tag: [commit hash after push]

1. Completion status, incomplete items, and blockers

Completed for this milestone:

- Development environment: Python 3.12 or later managed by uv, MySQL 8.4 running either from docker-compose.yml (not executed, Docker is not installed on the development machine) or from scripts/development/local_mysql.sh (a private mysqld without Docker), Node with pnpm for the frontend.
- Repository structure with backend, frontend, database, docs, and scripts directories (section 2).
- README setup draft: README.md at the repository root and docs/development/setup-guide.md.
- MySQL connection evidence (section 2): databases and the least-privilege user created by scripts/setup/init_databases.sh, Alembic migration applied, /api/v1/health responding.
- Schema and migration draft: Alembic revision 3a5e982c3679 (backend/migrations/versions/20261104_0106_create_manufacturing_schema.py) creating all 17 tables; a generated DDL snapshot in database/schema/schema.sql.
- Seed plan (section 3).

Incomplete or in progress:

- The frontend is implemented (React 19, TypeScript, Vite 7, Tailwind 4, React Router 7, TanStack Query 5, Recharts 3) with screens for Login, Overview, Machines, Orders, Batches, Inspections, Traceability, Investigations list, Investigation detail (case board with evidence cards and show-the-SQL), Analytics (charts plus SQL Explorer), and Simulation. Nine screenshots were captured from the running application on 2026-11-03 (docs/media/screenshots); no video has been recorded.
- Automated tests and the GitHub Actions workflows exist: `uv run pytest` last reported 100 passed (95 percent coverage of the backend `app` package) and `pnpm test` 39 passed in 13 files; `.github/workflows/` holds `backend-ci.yml`, `frontend-ci.yml`, and `docs-lint.yml`. All three workflows passed on the first push to GitHub on 2026-11-04 (Backend CI run 41779775539), and the test matrix (docs/testing/test-matrix.md) records which scenarios are only partly covered.
- The Docker path (docker-compose.yml with scripts/setup/init_databases.sh mounted as the init script) has not been executed, because Docker is not installed on the development machine; the local MySQL path (scripts/development/local_mysql.sh) is the one that has been run.
- [List any item that was planned for Week 9 and is not finished.]

Blockers: [describe any blocker, or write "none"].

2. Required technical artifacts and repository commit or tag

Repository structure:

Path	Contents
backend/app/api	FastAPI routers, dependencies, error handlers (31 endpoint paths; docs/api/openapi.json)
backend/app/services	Application services, one per workflow
backend/app/domain	State machines and quality rules (no database dependency)
backend/app/repositories	SQL file loader and query runner
backend/app/simulation	Deterministic virtual factory engine and fault models
backend/app/investigation	Six investigation rules and the evidence engine
backend/app/models	SQLAlchemy models, one class per table
backend/migrations	Alembic environment and versions
database/schema, seeds, queries	DDL snapshot, seed SQL and demo scenario, twelve query files

Path	Contents
frontend	React 19, TypeScript, Vite, Tailwind, Recharts, React Router, TanStack Query (implemented; see docs/architecture/repository-structure.md)
docs	architecture, database, simulation, investigation, requirements, testing, development, api, ai_evidence, milestones
scripts/setup, scripts/development	Database initialization, local MySQL, schema export, reset, OpenAPI export

MySQL connection evidence to include in the PDF:

- [Screenshot or terminal capture of scripts/development/local_mysql.sh status, or docker compose ps, showing the MySQL 8.4 server running.]
- [Terminal capture of uv run alembic upgrade head followed by uv run alembic current showing revision 3a5e982c3679.]
- [Terminal capture or browser capture of GET /api/v1/health returning a successful response.]
- [Output of SHOW TABLES on the factory_sherlock database listing the 17 tables.]

Repository commit or tag: [commit hash after push].

3. Decisions, alternatives, assumptions, constraints, and rationale

Decision	Alternatives considered	Rationale
Least-privilege application user created by scripts/setup/init_databases.sh with privileges only on the application and test schemas	Connect as root; one shared user for everything	The application cannot create users, grant privileges, or touch other schemas; the same script runs under docker-compose and under the local mysqld script
Two databases, factory_sherlock and factory_sherlock_test, selected by APP_ENV	Reuse the development database for tests	Tests can drop and recreate tables without touching development data
Alembic migrations as the schema source of truth, with database/schema/schema.sql exported by scripts/development/export_schema.sh	Hand-written DDL only	Migrations are versioned and reversible; the exported snapshot lets the schema be read without running the application
Seed reference data as SQL with INSERT ... ON DUPLICATE KEY UPDATE keyed on natural unique codes	Python inserts only; TRUNCATE and reload	Re-running the seed never duplicates rows, and the SQL the instructor reads is the SQL that runs
Demo accounts created by app.seed from SEED_DEMO_PASSWORD, which must be set	Hard-coded demo password in the seed file	No credential appears in the repository; the seed refuses to run without the variable
MySQL strict mode (STRICT_TRANS_TABLES, NO_ZERO_DATE, NO_ZERO_IN_DATE, ERROR_FOR_DIVISION_BY_ZERO) set at server start	Server defaults	Invalid dates and out-of-range values are rejected rather than silently adjusted

Seed plan: (1) database/seeds/01_reference_data.sql loads three products, six specifications, three machines (CNC-01, CNC-02, CNC-03), five defect types, and nine material lots; (2) app.seed creates the admin, engineer, operator, and viewer accounts with example.com addresses; (3) app.seed --demo runs the scenario in database/seeds/demo_scenario.json (seed 42, twelve batches per machine, twenty units per batch, tool wear on CNC-03 from batch index 4, downtime on CNC-02 from batch index 6).

Assumptions and constraints: the local MySQL binds to 127.0.0.1 only; the root account is used only by setup scripts; the project must run without paid services.

4. AI help: what, why, accepted, revised, rejected, and verification

Tool: Claude Code (Anthropic), used for the setup scripts, the seed loader, and the demo account definitions. The student ran every script against MySQL 8.4 before keeping it.

Stage	Task	Recommendation	Decision	Verification
Seed loading	SQL loader in backend/app/seed.py	The first version split the seed file on semicolons and dropped the first statement because the file begins with a comment block	Revised: comment lines are removed before splitting, so a statement that follows a comment block is kept	Found by running the seed against MySQL 8.4 and observing that the products table stayed empty; after the fix the seed loads all reference rows and is idempotent on a second run

Stage	Task	Recommendation	Decision	Verification
Demo accounts	Email addresses for seeded accounts	Suggested addresses on a .local domain	Rejected: the Pydantic email validator treats .local as a reserved domain and refuses the value; replaced with example.com addresses	Confirmed by running the seed and the login endpoint against MySQL 8.4 with the example.com accounts

Lesson recorded: AI-generated loaders and fixtures look complete but must be executed end to end; both problems were only visible at run time.

5. Lessons learned at this milestone

- A setup script that is used by both docker-compose and the local mysqld path must not depend on environment variables that the MySQL client also reads (MYSQL_HOST, MYSQL_TCP_PORT), or it silently connects elsewhere.
- Idempotent seeds (ON DUPLICATE KEY UPDATE on natural keys) remove an entire class of setup errors.
- [Add one lesson in your own words.]

6. Support or skill development needed for the next stage

[State any instructor support or skill development you need for Week 10, or write "None at this time".]

7. Next-step plan (Week 10)

1. Revise the ERD after the migration and record the normalization rationale in docs/database/database-design.md.
2. Load the demo scenario and run all twelve queries against it; capture results for the Week 10 report.
3. Document every constraint and index with its reason.
4. Capture initial CRUD evidence for products, machines, material lots, production orders, and batches through the API.
5. Continue recording AI interactions in docs/ai_evidence.

8. Personal reflection (Shritej Gunda)

[Write 150 to 250 words in your own voice. Replace each prompt.]

[What did you set up yourself, and what went wrong the first time?]

[Which part of the environment (MySQL, uv, migrations, seeds) do you now understand better, and why?]

[Describe one AI suggestion from this milestone that you revised or rejected and how you found the problem.]

[What is your plan for the Week 10 database and query defense?]

9. Personal-work summary

[List the artifacts you personally produced or verified for this milestone, one line each, with the file path or screen name.]